

Java Learning Center

No. 11 In Java Training & placement

Java Learning Center

@Annotation

Put Into Knowledge

Doc Version - 1.0

Author

Srinivas Dande

Kamlesh Kumar Singh



Introduction

- ♦ Annotations provides the special way of specifying metadata inside the java class instead of writing in XML file.
- ♦ When you are developing applications using Servlets, Struts, Hibernate, Spring, EJB etc. you need to write many XML documents which are specific to corresponding framework or technology.
- ♦ If you are specifying metadata inside the XML file then internally *SAX Parser* will be used to read the information from XML file and to store in main memory.
- ♦ With Java 5, you can avoid this XML files by specifying the metadata related to Servlets, Struts, Hibernate, Spring in the class itself with the help of Annotation i.e annotation is alternative to XML and provides another way of representing meta-data.
- ♦ There are two types of Annotations:
 - Built-in Annotations
 - User-defined Annotations

Built-in Annotations

- ♦ Following built-in annotations are provided from Java 5:
 - `@Override`
 - `@Deprecated`
 - `@SuppressWarnings`etc.
- ♦ Following built-in annotations are provided from Servlet 3:
 - `@WebServlet`
 - `@WebFilter`
 - `@WebListener`etc.
- ♦ Following built-in annotations are provided from EJB 3:
 - `@Stateless`
 - `@Stateful`
 - `@Remote`
 - `@Entity`etc.
- ♦ Following built-in annotations are provided from Spring 2.5: .
 - `@Autowired`
 - `@Transactional`
 - `@Aspect`etc.

@Override

- The @Override annotation can be applied for the methods only.
- It is used by the Java compiler to check if the annotated method really overrides a method of an interface or the extended class.

*Compiler will check it override or not
really it is override or not*

@Deprecated

- The @Deprecated annotation can be used on a fields, methods, constructors or classes.
- It is used by the Java compiler to check whether the element is outdated and should not be used anymore.
- Adding @Deprecated to the class does not deprecate automatically all its fields and methods.

out of date information

@SuppressWarnings

- The @SuppressWarnings annotation can be used on a fields, methods, constructors or classes.
- It just tells compiler to ignore specific warnings compiler produces.

hide that warning message

Lab1380.java

```
import java.util.Date;
public class Lab1380 {
    public static void main(String[] args) {
        Date dt = new Date();
        System.out.println(dt.getDate());
        System.out.println(dt.getMonth());
        System.out.println(dt.getYear());
    }
}
```

@Deprecated warning

Lab1381.java

```
import java.util.Date;
public class Lab1381 {
    public static void main(String[] args) {
        Student st = new Student("Sri");
        System.out.println(st);
        Student st1 = new Student(99,"Sri");
        System.out.println(st1);
    }
}
```

```
class Student{
    int sid;
    String name;
    Student(int sid,String name){
        this.sid = sid;
        this.name = name;
    }
    @Deprecated
    Student(String name){ }
    public String toString() {
        return sid+"\t"+name;
    }
}
```




Java Learning Center

No. 1 In Java Training & placement

Lab1382.java

```
public class Lab1382{
    public static void main(String[] args) {
        Employee em = new Employee(99, "Sri");
        System.out.println(em);
    }
}

class Employee {
    int eid;
    String name;
    Employee(int eid, String name) {
        this.eid = eid;
        this.name = name;
    }
    public String toString() {
        return eid + "\t" + name;
    }
}
```

no getting any error messages

Lab1383.java

```
public class Lab1383{
    public static void main(String[] args) {
        Employee em = new Employee(99, "Sri");
        System.out.println(em);
    }
}

class Employee {
    int eid;
    String name;
    Employee(int eid, String name) {
        this.eid = eid;
        this.name = name;
    }
    @Override
    public String toString() {
        return eid + "\t" + name;
    }
}
```

*it will check whether it is available or not
get error message*

Lab1384.java

```
public class Lab1384{
    public static void main(String[] args) {
        Employee em = new Employee(99, "Sri");
        System.out.println(em);
    }
}
```

```
class Employee {
    int eid;
    String name;
    Employee(int eid, String name) {
        this.eid = eid;
        this.name = name;
    }
    @Override
    public String toString() {
        return eid + "\t" + name;
    }
}
```

*Method is inherited so we
getting error.*

Lab1385.java

```
public class Lab1385{
    import java.util.Date;
    public class Lab {
        public static void main(String[] args) {
            Date dt = new Date();
            System.out.println(dt.getDate());
        }
    }
}
```

Lab1386.java

```
public class Lab1386{
    import java.util.Date;
    public class Lab {
        @SuppressWarnings("deprecation")
        public static void main(String[] args) {
            Date dt = new Date();
            System.out.println(dt.getDate());
        }
    }
}
```

*This type of warning
messages will be
hide*

Lab1387.java

```
public class Lab1387 {
    public static void main(String[] args) {
        @SuppressWarnings("unused")
        int ab = 10;
        System.out.println("Main Completed");
    }
}
```

Lab1388.java

```
public class Lab1388 {
    @SuppressWarnings("unused")
    public static void main(String[] args) {
        int ab = 10;
        System.out.println("Main Completed");
    }
}
```

Lab1389.java

```
import java.util.ArrayList;
public class Lab1389 {
    @SuppressWarnings({ "unused", "static-
access", "rawtypes" })
    public static void main(String[] args) {
        int ab = 10;
    }
}
```

```
Hello h = new Hello();
System.out.println(h.a);
ArrayList al=new ArrayList();
}
}
class Hello{
    static int a=90;
}
```

User-defined Annotations

- Following are three main tasks in Annotations implementation and uses.
 1. Define the Annotation.
 2. Provide the APT(Annotation Processing Tool)
 3. Use the Annotation
- Usually technology or frameworks vendors are responsible for defining the Annotation and providing the Annotation Processing Tools.
- As a developer, you are just responsible to use the Annotations.
- You cannot define constructors, initialisation blocks, methods within the annotation.
- You can define classes, interfaces, enums or annotations as the member of annotation.
- All the annotations are subtype of java.lang.annotation.Annotation interface.

Define the Annotation

Syntax:

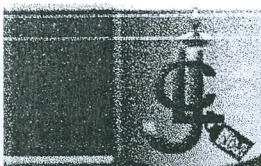
```
@interface Anno_Name{
    datatype member_name();
    datatype member_name();
    .....
}
```

Ex:

```
@interface
Author{
    String aname();
    int age();
}
```

Usage:

```
@Author(aname="Sri",age=36)
class Hello{
    ...
}
```

Types of Annotations

1. Marker Annotations
2. Single Valued Annotations
3. Multi Valued Annotations

Marker Annotations

- ♦ Annotation which is defined without any members are called as Marker Annotation.

Ex: <code>@interface JLCCloneable{ }</code>	Usage: <code>@JLCCloneable class Hello{ ... }</code>
--	---

Single Valued Annotations

- ♦ Annotations which are defined with only one member are called as single valued Annotations.

Ex: <code>@interface Author{ String value(); }</code>	Usage: <code>@Author(value="sri") class Hello{ }</code>	Usage: <code>@Author("sri") class Hello{ }</code>
Ex: <code>@interface Author{ String name(); }</code>	Usage: <code>@Author(name="sri") class Hello{ }</code>	Usage: <code>@Author("sri") //Invalid class Hello{ }</code>

Multi Valued Annotations:

- ♦ Annotations which are defined with two or more members are called as Multi Valued Annotations.

Ex: <code>@interface Employee{ int eid(); String ename(); long phone(); }</code>	Usage: <code>@Employee(eid=101,ename="Sri",phone=999999) class Hello{ ... }</code>
---	---

Default Valued Annotations:

- ♦ Annotations which are defined with default value for the members of annotation are called as Default Valued Annotations.

Ex: <code>@interface Author{ String name() default "Sri"; }</code>	Usage: <code>@Author(name="Nivas") class Hello{ }</code>	Usage: <code>@Author class Hello{ }</code>
---	--	---

- ♦ Till now we have seen how to define annotation and how to use the annotation.
- ♦ Now you have to develop ANNOTATION PROCESSING TOOL.
- ♦ You need the following to develop APT:
 - Meta Annotations
 - Reflection API related to Annotation

Meta Annotations

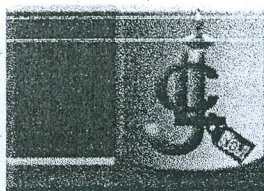
- ♦ Following are the Meta annotations provided in java.lang.annotation package:
 - @Retention
 - @Target
 - @Inherited

@Retention

- ♦ It will be used to specify the scope of user-defined annotation i.e visibility of the user-defined annotation.
- ♦ Scope of annotation is also called as RetentionPolicy.
- ♦ You need to use the following constants of java.lang.annotation.RetentionPolicy enum to specify the scope of annotations or RetentionPolicy of annotations.

SOURCECLASSRUNTIME
- ♦ CLASS is the default Retention Policy.

SOURCE	When the Retention Policy is SOURCE then annotations will be available only in source code and will not be available in Byte Code and Native Code. Usage: @Retention(RetentionPolicy.SOURCE) @interface Author{ }
CLASS	When the Retention Policy is CLASS then Annotations will be available only in source code and Byte Code and will not be available in and Native Code. Usage: @Retention(RetentionPolicy.CLASS) @interface Author{ }
RUNTIME	When the RetentionPolicy is RUNTIME then Annotations will be available in source code, Byte Code and Native Code. Usage: @Retention(RetentionPolicy.RUNTIME) @interface Author{ }



2) @Target

- ♦ This annotation is used to specify the Target, i.e to specify whether the annotation should be used for class or method or field or constructor or all.
- ♦ You need to use the following constants of java.lang.annotation.ElementType enum to specify the target of annotations.

CONSTRUCTOR

FIELD

LOCAL_VARIABLE

METHOD

PACKAGE

PARAMETER

TYPE

etc

- ♦ When you are not specifying target then that annotation can be used any where.

3) @Inherited

- ♦ When you specify any annotation for the class then that annotation will not be inherited in the subclass by default.
- ♦ If you want to inherit the super class annotations to the subclass then annotation definition must be marked with @Inherited.

Reflection API related to Annotation

- ♦ Following are the methods provided in java.lang.Class:
 - public boolean isAnnotation();
 - public Annotation getAnnotation(class);
 - public boolean isAnnotationPresent(class);
 - public Annotation[] getAnnotations();

Lab1390.java

```
import java.lang.annotation.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@interface JLCCloneable { }
```

```
@JLCCloneable
```

```
class Emp {
```

```
int eid;
```

```
String ename;
```

```
Emp(int eid, String ename) {
```

```
this.eid = eid;
```

```
this.ename = ename;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
return eid + "\t" + ename;
```

```
}
```

```
public Emp getClonedObject() throws
```

```
CloneNotSupportedException {
```

```
boolean b1 =
```

```
this.getClass().isAnnotationPresent(JLCCloneable.class);
```

```
if (b1) {
```

```
return new Emp(this.eid, this.ename);
```

```
}
```

```
throw new CloneNotSupportedException("Emp class not  
using JLCCloneable");
```

```
}}
```

```
public class Lab1390 {
```

```
public static void main(String[] args) throws
```

```
CloneNotSupportedException {
```

```
Emp emp = new Emp(99, "Srinivas");
```

```
System.out.println(emp);
```

```
Emp emp2 = emp.getClonedObject();
```

```
System.out.println(emp2);
```

```
System.out.println(emp == emp2);
```

```
}}
```


Lab1391.java

```
import java.lang.annotation.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@Target(ElementType.TYPE)
```

```
@interface Table {  
    String value();  
}
```

```
@Table("stud_table")  
class Stud { }
```

```
public class Lab1391 {  
    public static void main(String[] args) {  
        Stud st=new Stud();  
        Class cl=st.getClass();  
        boolean pre=cl.isAnnotationPresent(Table.class);  
        if(pre){  
            Table tb=(Table)cl.getAnnotation(Table.class);  
            String tnm=tb.value();  
            System.out.println("Table Name :"+tnm);  
        }  
        else{  
            System.out.println("Table Annotation is not present");  
        }  
    }  
}
```

Lab1392.java

```
import java.lang.annotation.*;
```

```
import java.lang.reflect.Field;
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
@Target(ElementType.FIELD)
```

```
@interface Column {  
    String name();  
    boolean charType() default false;  
}
```

```
class Faculty {  
    @Column(name="facultyId",charType=true)  
    int id;  
    @Column(name="facultyName")  
    String name;  
    long phone;  
}
```

```
public class Lab1392 {  
    public static void main(String[] args) {  
        Faculty f1=new Faculty();  
        Class cl=f1.getClass();  
        Field[] flds=cl.getDeclaredFields();  
        for (Field fld : flds) {  
            boolean pre=fld.isAnnotationPresent(Column.class);  
            if(pre){  
                Column col=(Column)fld.getAnnotation(Column.class);  
                String cnm=col.name();  
                boolean cty=col.charType();  
                System.out.println(fld.getName()+"\t"+cnm+"\t"+cty);  
            }else{  
                System.out.println("For "+fld.getName()+" Column  
annotation is not present");  
            }  
        }  
    }  
}
```

Lab1393.java

```
@Table(name = "studTable")
```

```
class JlcStudent {
```

```
    @Column(name = "sid",charType=false)
```

```
    int studId;
```

```
    @Column(name = "sfee",charType=false)
```

```
    float studFee;
```

```
    long phone;
```

```
    @Column(name = "sname")
```

```
    String studName;
```

```
}
```

```
@Table(name = "empTable")
```

```
class JlcEmployee {
```

```
    @Column(name = "eid",charType=false)
```

```
    int empld;
```

```
    @Column(name = "eage",charType=false)
```

```
    int age;
```

```
    @Column(name = "ename")
```

```
    String empName;
```

```
    @Column(name = "ephone",charType=false)
```

```
    long empPhone;
```

```
}
```



```
public class Lab{
public static void main(String[] args) throws
Exception {
JlcEmployee emp = new JlcEmployee();
emp.empId = 99;
emp.empName = "Srinivas";
emp.empPhone = 65799990;
JlcJdbcTemplate res = new JlcJdbcTemplate();
res.save(emp);
```

```
JlcStudent stud = new JlcStudent();
stud.studId = 7676;
stud.studName = "Nivas";
stud.phone = 9590712983L;
stud.studFee = 27000.0F;
res.save(stud);
}
}
```

JlcJdbcTemplate.java

```
import java.lang.annotation.*;
import java.lang.reflect.*;
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
@interface Table {
String name();
}
```

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
@interface Column {
String name();
boolean charType() default true;
}
```

```
class JlcJdbcTemplate {
public void save(Object obj) throws Exception
{
boolean tabPresent =
obj.getClass().isAnnotationPresent(Table.class);
if (tabPresent) {
Table tab =
obj.getClass().getAnnotation(Table.class);
String tableName = tab.name();
String qry = "insert into " + tableName;
StringBuffer cols = new StringBuffer(" ");
StringBuffer values = new StringBuffer("
values (");
```

```
Field fs[] = obj.getClass().getDeclaredFields();
for (int i = 0; i < fs.length; i++) {
Field f1 = fs[i];
boolean colPresent =
f1.isAnnotationPresent(Column.class);
if (colPresent) {
Column col= f1.getAnnotation(Column.class);
String colName=col.name();
boolean isChar=col.charType();
cols.append(colName + ","");
String type = f1.getType().getSimpleName();

String val = f1.get(obj).toString();
if (!isChar) {
values.append(val);
values.append(",");
} else {
values.append("''");
values.append(val);
values.append(",");
} } }
qry = qry + cols.substring(0, cols.length() - 1) + ")" +
values.substring(0, values.length() - 1) + " ";
System.out.println(qry);
//Code to Store in DB
} else {
String cName = obj.getClass().getName();
throw new RuntimeException(cName + " is not
annotated with @Table");
}
}
}
```

9672678755